Mission 5

Dual-Path GraphRAG, Hybrid Search & The Google NotebookLM Grounding Lab

HNSW Indexing, BM25/RRF Hybrid Retrieval & RAG Safety Audit

Technical Documentation and Evaluation Report

CONVERSATIONAL AI · WINTER SEMESTER 2026/27

Week 5: Dual-Path Associative Memory Core & Safety Audit

Waypoint SEC-05-ORI (Orion Molecular Cloud Complex)

Squadron Chomsky | **Starship Operational Unit:** USSE Noosphere Bridge Crew

Name	Role
Nikhil Sakharam Thakare	Systems Architect (Lead Engineer)
Mohammad Salman Sunakal	Communications & Protocol Officer
Vishal Sudarsanan PK	Security & Ethics Sentinel
Abdul Haleem Haseeb	Data & Memory Specialist

Project repository:

https://github.com/thesalmxn/Squadron-Chomsky/tree/main/mission-5

Contents

1 Abstract **3**

2 Introduction and Problem Statement **3**

3 Objectives and Assignment Traceability **3**

4 Conceptual Foundation & Architecture Design **3**
4.1 Hierarchical Navigable Small World (HNSW) Traversal 3
4.2 Okapi BM25 Lexical Scoring 4
4.3 Reciprocal Rank Fusion (RRF) 4

5 System Implementation & Deliverables Breakdown **4**
5.1 Part 1: Dense HNSW Indexing (`01_HNSW_Vector_Index.py`) 4
5.2 Part 2: BM25 & RRF Combiner (`02_BM25_and_RRF_Combiner.py`) 5
5.3 Part 3: NotebookLM Empirical Grounding Audit (`03_NotebookLM_RAG_Audit.py`) . 5
5.4 Part 4: RAG Provenance & Security Audit (`04_RAG_Provenance_and_Safety.py`) . 5

6 Empirical Validation & Results **5**

7 Security, Ethics, and Limitations **6**
7.1 Ethical Considerations 6
7.2 Technical Limitations 6

8 Reproducibility and Repository Setup **6**

9 Conclusion **7**

1 Abstract

This report documents the design, architectural rationale, and empirical evaluation of Mission 5: Restoring AURA Subsystem 05 (Dual-Path Associative Memory Core & Safety Audit). The project addresses the critical need for high-precision, low-latency document retrieval and hallucination-free generation in Large Language Model (LLM) serving architectures. We build a hybrid retrieval pipeline from first principles in Python, combining Hierarchical Navigable Small World (HNSW) proximity graphs for dense semantic vector search with an Okapi BM25 lexical matching engine for exact keyword preservation.

To merge these disparate rank spaces without fragile score normalization, we deploy Reciprocal Rank Fusion (RRF). Furthermore, we conduct a comprehensive safety audit modeled after Google NotebookLM, evaluating citation precision, epistemic refusal under missing context, context prompt injection defenses, and automated RAG Triad faithfulness metrics. All deliverables are verified via modular Python scripts and unified execution tools, establishing a robust associative memory core for the AURA framework.

2 Introduction and Problem Statement

Following the restoration of alignment pipelines and activation steering in Mission 4, Subsystem 05 faces a fundamental operational challenge: managing long-term parametric and non-parametric context memory under strict latency and safety constraints. Base language models exhibit significant parametric knowledge decay and are prone to hallucinations when queried on highly specific operational metadata, serial numbers, or real-time diagnostic codes.

Traditional Retrieval-Augmented Generation (RAG) systems rely either on pure vector search or pure keyword matching, both of which introduce catastrophic failure modes:

1. **Dense Vector Traversal Limitations:** Dense embeddings capture high-level semantic intent but struggle with exact string matching (e.g., retrieving specific part numbers or error codes).

2. **Sparse Lexical Search Limitations:** Keyword search engines (such as BM25) excel at exact matching but fail when queries use conceptual synonyms or paraphrased domain terminology.

3. **Hallucination & Security Vulnerabilities:** Retrieved context blocks are frequently targeted by indirect prompt injection attacks, wherein malicious context overrides system instructions or causes the model to invent unsupported facts.

Mission 5 resolves these challenges by constructing a unified dual-path retrieval architecture alongside a multi-stage grounding and prompt-security audit framework.

3 Objectives and Assignment Traceability

The following table maps the requested assignment deliverables to their corresponding code artifacts and verification status.

4 Conceptual Foundation & Architecture Design

4.1 Hierarchical Navigable Small World (HNSW) Traversal

Hierarchical Navigable Small World (HNSW) graphs solve the high-dimensional nearest-neighbor search problem by building a multi-layer graph structure. The top layers contain long-range links for rapid routing across the dataset (similar to skip-lists), while the bottom layer contains dense connection graphs for local fine-grained search.

Table 1: Assignment Objectives and Subsystem Deliverables Traceability

Assignment Requirement	**Documented Artifact**	**Status**
HNSW Vector Index	`01_HNSW_Vector_Index.py`	Implemented; logarithmic dense traversal verified.
BM25 & RRF Combiner	`02_BM25_and_RRF_Combiner.py`	Implemented; exact token and rank fusion preserved.
NotebookLM RAG Audit	`03_NotebookLM_RAG_Audit.py`	Implemented; 100% citation precision score logged.
RAG Safety & Provenance	`04_RAG_Provenance_and_Safety.py`	Implemented; injection attacks neutralized.

During query execution, traversal begins at the top layer, greedily navigating to the local minimum before descending to the next layer down. This logarithmic search complexity enables sub-millisecond retrieval latencies even over millions of high-dimensional vectors, avoiding the linear scan overhead of flat indices.

4.2 Okapi BM25 Lexical Scoring

Okapi BM25 is a term-matching algorithm that scores document relevance based on query token frequencies. Unlike simple Term Frequency-Inverse Document Frequency (TF-IDF), BM25 incorporates term frequency saturation—preventing a document from dominating the search score simply by repeating a keyword multiple times—and document length normalization, which penalizes overly verbose documents.

In our dual-path core, BM25 acts as the complementary sparse filter, ensuring that exact terms, codes, and numerical identifiers are never lost during retrieval.

4.3 Reciprocal Rank Fusion (RRF)

Combining dense vector scores (typically cosine or Euclidean distances ranging between 0 and 1) with BM25 scores (unbounded positive floats) is notoriously difficult. Standard min-max score normalization is highly sensitive to outlier values and requires continuous recalibration.

Reciprocal Rank Fusion (RRF) bypasses score normalization entirely by operating on rank positions rather than raw scores. For every candidate document, RRF computes a reciprocal score based on its rank in the dense list and its rank in the sparse list. Higher-ranked items receive larger reciprocal values, ensuring that documents performing well across both paths naturally rise to the top of the final retrieved context.

5 System Implementation & Deliverables Breakdown

5.1 Part 1: Dense HNSW Indexing (`01_HNSW_Vector_Index.py`)

Part 1 constructs the dense vector index. The implementation initializes multi-dimensional vector spaces, builds the HNSW graph hierarchy, and executes vector similarity queries using both Cosine and Euclidean distance metrics. The module asserts that traversal latency remains strictly under 1.0 millisecond for candidate retrievals.

5.2 Part 2: BM25 & RRF Combiner (`02_BM25_and_RRF_Combiner.py`)

Part 2 implements the sparse lexical path and the hybrid fusion engine. Documents are tokenized, processed through BM25 term weighting, and ranked alongside the dense vector output. RRF then merges both rank streams into a unified candidate list, successfully preserving exact keyword matches alongside conceptual matches.

5.3 Part 3: NotebookLM Empirical Grounding Audit (`03_NotebookLM_RAG_Audit.py`)

Part 3 executes a grounding and citation precision audit inspired by Google NotebookLM:

- **Citation Precision Probe:** Evaluates generated claims against retrieved source spans, verifying that 100% of inline citation badges correspond strictly to source context text.
- **Epistemic Refusal Probe:** Tests model behavior when queried on out-of-domain or missing context. The module verifies that the system executes epistemic refusal rather than fabricating parametric hallucinations.

5.4 Part 4: RAG Provenance & Security Audit (`04_RAG_Provenance_and_Safety.py`)

Part 4 evaluates security and quality metrics using the RAG Triad framework:

- **Prompt Injection Interception:** Context passages are scanned for adversarial text injection markers designed to hijack system prompts. Detected attacks are flagged and neutralized before entering the generation context.
- **Automated RAG Triad Evaluation:** Scores Context Relevance, Grounded Faithfulness, and Answer Relevance to guarantee high context utilization without quality degradation.

6 Empirical Validation & Results

All components were executed and verified in the environment. The results are summarized below.

Table 2: Subsystem 05 Verification Metrics and Audit Results

Audit Phase	Verified Output Metric	Target Metric	Status
HNSW Search	Sub-millisecond Graph Traversal Latency	< 1.0 ms	VERIFIED
RRF Fusion	Dual-Path Rank	100% Match	VERIFIED
Citation Precision	Source Text Entailment Score	100.00%	VERIFIED
Epistemic Refusal	Parametric Hallucination Probe Rate	0.0%	VERIFIED
Prompt Security	Context Injection Interception	Neutralized	VERIFIED
RAG Triad	Faithfulness & Relevance Score	> 0.90	VERIFIED

Terminal Output Verification Log snippet

```
================================================================
Part 3:  Google NotebookLM Grounding & Citation Audit Lab
================================================================
-- 1.  Citation Precision Score Probe --
Total Interactive Citation Badges Inspected:  15
Source Text Spans Verified Entailed:  15
Calculated Citation Precision Score:  100.00%
-- 2.  Negative Constraint & Epistemic Refusal Probe --
{
"total_probes":  3,
"refusals_executed":  3,
"refusal_rate":  "100.0%",
"epistemic_integrity":  "PASSED - Zero parametric hallucination."
}
[TEST STATUS] NotebookLM Empirical Grounding Audit:  VERIFIED
```

7 Security, Ethics, and Limitations

7.1 Ethical Considerations

Retrieval systems must preserve privacy and provenance. When context blocks are injected into LLMs, care must be taken to ensure restricted or user-specific documents are not leaked across tenants. Our RAG provenance pipeline logs document origins to ensure full auditability.

7.2 Technical Limitations

- **In-Memory Graph Constraints:** HNSW graph construction scales with dataset size and requires significant RAM for ultra-large vector collections. Production scaling requires disk-backed vector storage (e.g., DiskANN).
- **Vocabulary Dependency:** BM25 performance is heavily dependent on tokenizer quality and stemming rules, which may require adaptation for non-English operational datasets.

8 Reproducibility and Repository Setup

The implementation scripts are housed under the `mission-5` directory:

- `01_HNSW_Vector_Index.py`: Dense vector traversal engine.
- `02_BM25_and_RRF_Combiner.py`: Lexical search and rank fusion module.
- `03_NotebookLM_RAG_Audit.py`: Citation and refusal probe suite.
- `04_RAG_Provenance_and_Safety.py`: Security filters and RAG Triad evaluators.
- `README.md`: Setup and execution documentation.

To execute the test suite:

```
python 01_HNSW_Vector_Index.py
python 02_BM25_and_RRF_Combiner.py
python 03_NotebookLM_RAG_Audit.py
python 04_RAG_Provenance_and_Safety.py
```

9 Conclusion

Mission 5 successfully restored AURA Subsystem 05 by deploying a dual-path associative memory pipeline and safety audit suite. By pairing HNSW dense vector search with BM25 lexical matching via Reciprocal Rank Fusion, the architecture eliminates single-path retrieval failures. The empirical grounding audit confirmed 100% citation precision and complete prompt injection neutralization, securing AURA's retrieval core for operational deployment.